

LAB-1

Question 1

Create a NumPy array containing the integers from 1 to 20. Perform the following operations:

1. Display the array.
2. Find the shape of the array.
3. Find the maximum and minimum values.
4. Calculate the mean and standard deviation.
5. Reshape the array into a 4×5 matrix.
6. Display the second row of the matrix.
7. Display the third column of the matrix.

CODE:

```
import numpy as np

arr = np.arange(1,21)

matrix = np.reshape(arr,(4,5))

mean = arr.mean()

std = arr.std()

row_2 = matrix[1,:]

column_3 = matrix[:,2]

print("array : ", arr)

print("shape :", arr.shape)

print("minimum element in array: ", arr.min())

print("maximum element in array: ", arr.max())

print("4_5_matrix :\n", matrix)

print("mean : ", mean)

print("standard deviation : ",std)

print("second row :",row_2)

print("third column :",column_3)
```

OUTPUT:

```
array : [ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]
shape : (20,)
minimum element in array:  1
maximum element in array: 20
4_5_matrix :
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]]
mean : 10.5
standard deviation :  5.766281297335398
second row : [ 6  7  8  9 10]
third column : [ 3  8 13 18]
```

Question 2

Create two NumPy arrays:

A = [10, 20, 30, 40, 50]

B = [5, 10, 15, 20, 25]

Perform:

1. Addition of A and B.
2. Subtraction of B from A.
3. Element-wise multiplication.
4. Element-wise division.
5. Find the sum of all elements in A.

CODE:

```
import numpy as np

A = np.arange(10,60,10)

B = np.arange(5,30,5)

print("A:",A)

print("B:",B)

add_A_B = A + B

diff_A_B = A - B

diff = np.setdiff1d(A,B)

mul = A*B
```

```

div = A/B

sum_A = A.sum()

print("addition of A and B :", add_A_B)

print("difference of A and B: ", diff_A_B)

print("multion od A and B: ", mul)

print("division of A and B:", div)

print("sum of elements in A: ", sum_A)

print(diff)

```

OUTPUT:

```

A: [10 20 30 40 50]
B: [ 5 10 15 20 25]
addition of A and B : [15 30 45 60 75]
difference of A and B: [ 5 10 15 20 25]
multion od A and B: [ 50 200 450 800 1250]
division of A and B: [2. 2. 2. 2. 2.]
sum of elements in A: 150
[30 40 50]

```

Question 3

Create a DataFrame using the following student

data:

Roll No	Name	Marks
101	Ravi	85
102	Priya	92
103	Kiran	78
104	Anu	88
105	Sita	95

Perform the following operations:

1. Display the DataFrame.
2. Display the first three records.
3. Find the average marks.
4. Find the student with the highest marks.

5. Find the student with the lowest marks.

6. Add a new column called "Grade" using:

- A for Marks ≥ 90
- B for Marks between 80 and 89
- C for Marks < 80

CODE:

```
import pandas as pd

data = {

    "roll": [101,102,103,104,105],

    "name": ["ravi","priya","kiran","anu","sita"],

    "marks": [85,92,78,88,95]

}

df = pd.DataFrame(data)

print(df)

print(df.head(3))

print("average marks:", df["marks"].mean())

print(df.loc[[df["marks"].idxmax()], ["name", "marks"]])

print(df.loc[[df["marks"].idxmin()], ["name", "marks"]])

def grade(marks):

    if marks>=90:

        return "A"

    elif 80 <=marks<=89:

        return "B"

    else:

        return "C"

df["grades"] = df["marks"].apply(grade)

print(df)
```

OUTPUT:

```
roll  name  marks
0   101   ravi    85
1   102  priya    92
2   103  kiran    78
3   104   anu    88
4   105   sita    95
roll  name  marks
0   101   ravi    85
1   102  priya    92
2   103  kiran    78
average marks: 87.6
name  marks
4   sita    95
name  marks
2   kiran    78
roll  name  marks  grades
0   101   ravi    85      B
1   102  priya    92      A
2   103  kiran    78      C
3   104   anu    88      B
4   105   sita    95      A
```

Question 4

Create a DataFrame containing the following information:

Product	Price	Quantity
Laptop	50000	10
Mouse	500	50
Keyboard	1200	30
Monitor	10000	15

Perform:

1. Calculate the total value of each product using: $\text{Total Value} = \text{Price} \times \text{Quantity}$
2. Add the Total Value column.
3. Find the product with the highest total value.
4. Calculate the overall inventory value.

CODE:

```
import pandas as pd
```

```
data = {  
    "product" : ["laptop","mouse","keyboard","monitor"],  
    "price" : [50000,500,1200,10000],  
    "quantity" : [10,50,30,15]  
}  
  
df = pd.DataFrame(data)  
  
print(df)  
  
df["total"] = df["price"]*df["quantity"]  
  
print(df)  
  
print("highest price product:",df.loc[df["total"].idxmax()])  
  
print("total value:", df["total"].sum())
```

OUTPUT:

```
   product  price  quantity  total  
0  laptop  50000         10  500000  
1   mouse    500         50   25000  
2 keyboard   1200         30   36000  
3  monitor  10000         15  150000  
highest price product: product    laptop  
price           50000  
quantity         10  
total           500000  
Name: 0, dtype: object  
total value: 711000
```

Question 5

Download the Iris dataset from Scikit-learn and perform the following:

1. Load the dataset into a Pandas DataFrame.
2. Display the first five records.
3. Find the number of rows and columns.
4. Display column names.
5. Check for missing values.
6. Generate summary statistics using describe().
7. Find the number of samples belonging to each class.

CODE:

```
from sklearn.datasets import load_iris

import pandas as pd

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

df["target"] = iris.target

print(df.head())

print("shape:", df.shape)

print(df.columns)

print(df.isnull().sum())

print(df.describe())

print(df["target"].value_counts())
```

OUTPUT:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

	target
0	0
1	0
2	0
3	0
4	0

shape: (150, 5)

Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
'petal width (cm)', 'target'],
dtype='object')

sepal length (cm)	0
sepal width (cm)	0
petal length (cm)	0
petal width (cm)	0
target	0

dtype: int64

	sepal length (cm)	sepal width (cm)	petal length (cm)	\
count	150.000000	150.000000	150.000000	
mean	5.843333	3.057333	3.758000	
std	0.828066	0.435866	1.765298	
min	4.300000	2.000000	1.000000	
25%	5.100000	2.800000	1.600000	
50%	5.800000	3.000000	4.350000	
75%	6.400000	3.300000	5.100000	
max	7.900000	4.400000	6.900000	

	petal width (cm)	target
count	150.000000	150.000000
mean	1.199333	1.000000
std	0.762238	0.819232
min	0.100000	0.000000
25%	0.300000	0.000000
50%	1.300000	1.000000
75%	1.800000	2.000000
max	2.500000	2.000000

target

0	50
1	50
2	50

Name: count, dtype: int64